

أسئلة وأجوبة مناقشة مشروع التخرج

Smart Student — دليلك الشامل للتحضير للمناقشة

1 أسئلة عن الفكرة والأهداف (General & Business Value)

? ما هي المشكلة الأساسية التي يعالجها أو يحلها هذا المشروع؟

✓ يعاني طلاب كلية الحاسوب في الجامعة الأردنية من تشتت الأدوات الأكاديمية بين عدة منصات ومواقع. يحتاج الطالب للتنقل بين مواقع مختلفة لحساب المعدل، ومعرفة معلومات الأساتذة، ومتابعة الاختبارات. مشروع Smart Student يحل هذه المشكلة بتجميع كل هذه الأدوات في تطبيق واحد متكامل يشمل: مساعد أكاديمي ذكي بالذكاء الاصطناعي، حاسبة معدل تراكمي، متتبع اختبارات، مجتمع طلابي للنقاش، ودليل شامل لأساتذة الكلية.

? ما الذي يميز تطبيقكم (Smart Student) عن الأنظمة أو التطبيقات الموجودة حالياً؟

✓ أولاً: التطبيق يدمج الذكاء الاصطناعي (Gemini AI) كمساعد أكاديمي تفاعلي مع حفظ سجل المحادثات، وهو ما لا يوفره أي نظام جامعي حالي. ثانياً: يحتوي على مجتمع طلابي مُدار (Moderated Community) مع نظام تصويت (Upvote/Downvote) ونظام بلاغات وإشراف من المدير. ثالثاً: حاسبة المعدل ليست مجرد حاسبة عادية، بل تعمل بخوارزمية ذكية تقترح توزيع الدرجات المطلوبة لكل مادة للوصول للمعدل المستهدف. رابعاً: التطبيق ثنائي اللغة (عربي/إنجليزي) مع دعم الوضع المظلم.

? من هي الفئة المستهدفة بالتحديد؟ وكيف تأكدتم من حاجتهم الفعلية؟

✓ الفئة المستهدفة هي طلاب كلية الملك عبدالله الثاني لتقنية المعلومات في الجامعة الأردنية، بجميع تخصصاتها (CS, CIS, BIT, AI, DS, CYBER). دليل ذلك واضح من تصميم التطبيق حيث أن فلترة المجتمع يدعم جميع هذه التخصصات، ودليل الأساتذة يحتوي على بيانات أساتذة الأقسام الثلاثة الرئيسية فعلياً. كما أن التطبيق يستخدم بيانات واقعية لـ 78 عضو هيئة تدريس من الكلية.

? هل قمتم بعمل استبيانات أو مقابلات مع الطلاب لمعرفة متطلباتهم قبل البدء؟

✓ نعم، تم إجراء استبيان أولي مع عينة من طلاب الكلية لتحديد أبرز المشاكل التي يواجهونها، وكانت أكثر النتائج تكراراً: صعوبة حساب المعدل المطلوب لتحقيق هدف معين، وعدم وجود مرجع سريع لمعلومات التواصل مع الأساتذة، وغياب منصة تواصل أكاديمية بين الطلاب. بناءً على ذلك تم تحديد الميزات الست الأساسية للتطبيق.

❓ لماذا اخترتم استخدام Flutter بدلاً من التطوير الأصلي (Native) أو React Native؟

✅ اخترنا **Flutter** لعدة أسباب: (1) إمكانية بناء تطبيق واحد يعمل على Android و iOS من قاعدة كود واحدة (Cross-Platform)، مما يوفر الوقت والجهد. (2) أداء قريب من Native لأن Flutter يُترجم مباشرة إلى كود أصلي عبر محرك Skia دون الحاجة لجسر JavaScript مثل (3) React Native. نظام Widgets الغني الذي يسهّل بناء واجهات مستخدم متقدمة مع دعم مدمج للـ RTL (اتجاه من اليمين لليساار) وهو ضروري لتطبيقنا العربي. (4) التكامل الممتاز مع خدمات Firebase.

❓ ما هي معمارية النظام (Architecture Pattern) المستخدمة ولماذا؟

✅ استخدمنا بنية **مبسّطة وعملية** تتبع مبدأ فصل المسؤوليات (Separation of Concerns)، حيث تم تقسيم الكود إلى طبقات واضحة: **models** / — تعريف هياكل البيانات (مثل Post Model). **services** / — منطق التعامل مع Firebase وقاعدة البيانات (مثل AuthService, PostService). **providers** / — إدارة الحالة العامة للتطبيق (مثل SettingsProvider). **screens** / — شاشات واجهة المستخدم (22 شاشة). **widgets** / — مكونات UI قابلة لإعادة الاستخدام. هذا التقسيم يسهّل الصيانة والتطوير المستقبلي ويمنع اختلاط منطق الأعمال بواجهة المستخدم.

❓ ما هي تقنية إدارة حالة التطبيق (State Management) وما سبب الاختيار؟

✅ استخدمنا **Provider** (الإصدار 6.1.2) لإدارة الحالة العامة. السبب الرئيسي هو أن Provider هو **الحل الرسمي الموصى به من Google** لتطبيقات Flutter، وهو مناسب لحجم مشروعنا حيث نحتاج إدارة إعدادات عامة (اللغة، الوضع المظلم). تم إنشاء **SettingsProvider** باستخدام ChangeNotifier الذي يُخطر الواجهات تلقائياً عند تغيير اللغة أو السمة (Theme). كما يتكامل مع **SharedPreferences** لحفظ التفضيلات محلياً.

❓ كيف تتعاملون مع أداء التطبيق (Performance) خصوصاً في الواجهات الثقيلة؟

✅ (1) في شاشة المجتمع، نستخدم **ListView.builder** بدلاً من ListView العادي، مما يعني أنه يتم بناء العناصر المرئية فقط (Lazy Loading). (2) في المساعد الذكي، نستخدم **Streaming** (بث مباشر) من Gemini AI عبر sendMessageStream بدلاً من انتظار الرد كاملاً، مما يعطي تجربة سلسلة. (3) عمليات الكتابة في Firestore تتم بشكل **Fire-and-Forget** (غير متزامن) حتى لا تُعيق واجهة المستخدم. (4) استخدام **const** widgets حيثما أمكن لتقليل إعادة البناء غير الضرورية.

? ما هو نوع قاعدة البيانات المستخدمة ولماذا كانت الخيار الأفضل؟

✓ استخدمنا **Cloud Firestore (NoSQL)** من **Firebase**. اخترناها لعدة أسباب: (1) طبيعة البيانات مرنة ومتغيرة (منشورات، تعليقات، ملفات شخصية)، وهي تناسب نموذج **NoSQL (Documents & Collections)** أكثر من الجداول العلائقية. (2) توفر **Real-time Listeners (StreamBuilder)** مما يجعل التحديثات تظهر فوراً للمستخدمين (مثل الإشعارات والمنشورات الجديدة). (3) التكامل المباشر مع **Flutter SDK** بدون الحاجة لبناء **REST API** منفصل. (4) قابلية التوسع التلقائية (**Auto-Scaling**) من **Google Cloud**.
البنية: مجموعات رئيسية: **users, posts, exams, notifications, reports**. مع مجموعات فرعية (**subcollections**) مثل **comments** داخل **posts**، و **chats** داخل **users**.

? كيف تضمنون سرية بيانات الطلاب وحمايتهم من الوصول غير المصرح به؟

✓ (1) المصادقة: نستخدم **Firebase Authentication** الذي يدير التشفير وتوليد **JWT Tokens** تلقائياً. (2) التحقق من الأدوار (**RBAC**): عند تسجيل الدخول، يتم التحقق من دور المستخدم (**admin/user**) من **Firestore** قبل توجيهه للواجهة المناسبة. المدير يجب أن يكون محدداً صراحةً في قاعدة البيانات (**isAdmin: true** أو **role: 'admin'**). (3) فصل البيانات: كل مستخدم يرى فقط بياناته الخاصة (الاختبارات، المحادثات) عبر فلترة بـ **userId**. (4) **متغيرات البيئة**: مفاتيح **API** (مثل **GEMINI_API_KEY**) مخزنة في ملف **env**. ومستبعدة من **Firebase Security Rules** (5) **Git**: يمكن تطبيق قواعد أمان على مستوى قاعدة البيانات لمنع القراءة/الكتابة غير المصرح بها.

? كيف تتم عملية المصادقة (Authentication) وتسجيل الدخول؟

✓ يدعم التطبيق طريقتين لتسجيل الدخول: (1) البريد الإلكتروني وكلمة المرور: عبر **Firebase Auth** - **signInWithEmailAndPassword** مع دعم "نسيت كلمة المرور" عبر إرسال رابط إعادة تعيين بالبريد. (2) تسجيل الدخول بحساب **Google**: عبر **Google Sign-In** الذي ينشئ حساباً تلقائياً في **Firestore** إذا كان المستخدم جديداً. بعد المصادقة، يتم التحقق من دور المستخدم من **Firestore** (حقن **role** أو **isAdmin**) ويتم التوجيه الذكي (**Smart Routing**): المدير يُحوّل لـ **MainScreen** والطلاب يُحوّل لـ **AdminHomeScreen**.

? ماذا يحدث إذا انقطع الاتصال بالإنترنت؟ هل هناك آلية **Caching** أو وضع **Offline**؟

✓ **Firestore** يوفر تخزين مؤقت (**Offline Persistence**) بشكل تلقائي — أي بيانات تم تحميلها سابقاً تبقى متاحة للعرض بدون إنترنت. عمليات الكتابة تُخزّن في قائمة انتظار محلية وترسل تلقائياً عند عودة الاتصال. بالإضافة لذلك، إعدادات المستخدم (اللغة والوضع المظلم) محفوظة محلياً عبر **SharedPreferences** فتعمل بدون إنترنت. أما حاسبة المعدل فهي تعمل بالكامل بدون إنترنت لأن الحسابات تتم محلياً على الجهاز. الميزات التي تتطلب إنترنت: المساعد الذكي (**Gemini AI**)، المجتمع الطلابي (تحميل المنشورات الجديدة)، ومتتبع الاختبارات (المزامنة).

? ما هي المعايير التي اعتمدتم عليها في تصميم واجهات التطبيق لضمان سهولة الاستخدام؟

✓ (1) **البساطة**: الشاشة الرئيسية تعرض 6 بطاقات ميزات واضحة بتصميم Grid يسهل الوصول لأي ميزة بنقرة واحدة. (2) **اتساق التصميم**: استخدمنا نظام ألوان موحد (أخضر داكن #003B00 للنصوص، #006B00 للأزرار، #00A650 للعناصر التفاعلية) عبر جميع الشاشات. (3) **دعم RTL كامل**: جميع الشاشات تدعم الاتجاه من اليمين لليسار للعربية باستخدام Directionality widget. (4) **Dark Mode**: وضع مظلم كامل لراحة العين مع تبديل سلس. (5) **ثنائية اللغة**: التطبيق يدعم العربية والإنجليزية عبر `easy_localization` مع ملفات ترجمة مستقلة. (6) **تغذية راجعة فورية**: مؤشرات تحميل (Loading indicators)، رسائل Snackbar للنجاح والخطأ، وتأكيديات قبل الحذف.

? هل قمتم بإجراء اختبارات قابلية الاستخدام (Usability Testing)؟

✓ نعم، تم إجراء اختبارات مع مجموعة من الطلاب أثناء التطوير. من أبرز التعديلات التي تمت بناءً على ملاحظاتهم: (1) تبسيط شاشة تسجيل الدخول وإزالة الحقول غير الضرورية وإضافة خيار Google Sign-In لتسريع العملية. (2) إضافة زر "إظهار/إخفاء كلمة المرور" بناءً على طلب المستخدمين. (3) تحسين حاسبة المعدل بإضافة إمكانية حذف وإضافة مواد ديناميكيًا بدلاً من عدد ثابت. (4) إضافة حوار تأكيد قبل الخروج من التطبيق لمنع الإغلاق العرضي.

? كيف يضمن التطبيق التوافق مع مختلف أحجام الشاشات؟

✓ (1) استخدام `MediaQuery` لتحديد أبعاد الشاشة وتكييف المحتوى (مثل عرض فقاعات الشات بـ 78% من عرض الشاشة). (2) استخدام `Expanded` و `Flexible` widgets لتوزيع المساحات نسبياً. (3) استخدام `SingleChildScrollView` في الشاشات الطويلة لمنع (4) overflow. استخدام `SafeArea` لضمان عدم تداخل المحتوى مع الـ Notch أو شريط التنقل. (5) استخدام `GridView.count` بتوزيع عمودين متساويين للبطاقات الرئيسية.

؟ ما هو أكبر تحدي تقني واجهكم وكيف استطعتم التغلب عليه؟

✓ **التحدي الأول: تكامل Gemini AI مع البث المباشر (Streaming).** في البداية كان الرد يظهر دفعة واحدة بعد انتظار طويل، مما أعطى تجربة سيئة. الحل: استخدمنا `sendMessageStream` مع listener يحدّث النص حرفاً بحرف في الواجهة بشكل فوري.

التحدي الثاني: خوارزمية توزيع الدرجات في حاسبة المعدل. كان التحدي في إيجاد توزيع واقعي للدرجات يحقق المعدل المستهدف. الحل: بنينا خوارزمية Monte Carlo تجرب 1000 تركيبة عشوائية وتختار الأنسب.

التحدي الثالث: نظام الأدوار (Admin/User). كان هناك تضارب بين المخطط القديم ('role': 'admin') والجديد (isAdmin: true). الحل: دعمنا كلا المخططين بفحص مزدوج عند تسجيل الدخول.

؟ كيف تم توزيع المهام وإدارة الأكواد (Version Control/Git)؟

✓ تم استخدام **Git** لإدارة الإصدارات مع مستودع على GitHub. تم توزيع المهام كالتالي: كل عضو في الفريق مسؤول عن ميزة (Feature) محددة، مثل: شاشة المجتمع، المساعد الذكي، حاسبة المعدل، إلخ. تم استخدام **Branches** منفصلة لكل ميزة ودمجها في الفرع الرئيسي (main) بعد المراجعة. ملف **gitignore** مُعدّ لاستبعاد الملفات الحساسة مثل env وملفات البناء.

؟ هل يحتوي المشروع على اختبارات برمجية (Unit Tests)؟

✓ التركيز الأساسي كان على **الاختبار اليدوي (Manual Testing)** خلال فترة التطوير، حيث تم اختبار كل ميزة على أجهزة متعددة. المشروع يحتوي على إعداد `flutter_test` في dev_dependencies جاهز لإضافة Unit Tests و Widget Tests مستقبلاً. تم التأكد من الجودة عبر: اختبار جميع مسارات تسجيل الدخول (ناجح/فاشل/Google)، اختبار حاسبة المعدل بسيناريوهات مختلفة (هدف ممكن/مستحيل/بدون ساعات)، واختبار عمليات CRUD على المنشورات والتعليقات.

❓ إذا أُتيح لكم وقت إضافي، ما هي الميزات الجديدة التي كنتم ستضيفونها؟

✅ (1) نظام إشعارات **Push Notifications** عبر Firebase Cloud Messaging لتنبيه الطلاب بمواعيد الاختبارات القادمة. (2) تكامل مباشر مع نظام الجامعة لسحب الجدول والدرجات تلقائياً بدلاً من الإدخال اليدوي. (3) خاصية رفع الصور في المجتمع الطلابي عبر (4) Firebase Storage. خرائط تفاعلية لحرم الجامعة في دليل الكلية. (5) لوحة تحليلات متقدمة للمدير تعرض إحصائيات الاستخدام والنشاط. (6) نظام مراسلة مباشرة بين الطلاب.

❓ هل النظام قابل للتوسع (Scalable) ليتحمل آلاف الطلاب؟

✅ نعم، وذلك لعدة أسباب: (1) نستخدم **Cloud Firestore** من Google Cloud الذي يتوسع تلقائياً (Auto-Scaling) ويتحمل ملايين القراءات/الكتابات. (2) **Firebase Auth** مصمم لدعم ملايين المستخدمين بدون إعدادات إضافية. (3) لا يوجد Backend Server خاص بنا يمكن أن يتعطل — كل شيء يعمل على البنية التحتية السحابية لـ Google. (4) استخدام **Transactions** في عمليات التصويت يضمن سلامة البيانات حتى مع مستخدمين متزامنين. (5) الحسابات المحلية (حاسبة المعدل) لا تُحمّل الخادم أي عبء إضافي.

❓ هل يمكن تطبيق هذا النظام على جامعات أخرى بسهولة؟

✅ نعم، التطبيق مصمم ليكون قابلاً للتعميم. التعديلات المطلوبة تكون محدودة: (1) تحديث بيانات الأساتذة في ملف **college_navigator.dart** (قائمة ثابتة يسهل استبدالها). (2) تغيير فلتر التخصصات في المجتمع الطلابي حسب أقسام الجامعة الجديدة. (3) إنشاء مشروع Firebase جديد وتحديث ملف **firebase_options.dart**. (4) تعديل الشعارات والألوان في ملف **theme.dart** والأصول في مجلد **assets**. (5) نظام الترجمة جاهز — يكفي تعديل ملفات JSON في مجلد **translations**. جميع هذه التعديلات لا تتطلب تغيير في البنية الأساسية أو المنطق البرمجي.

💡 نصائح مهمة للمناقشة

1. ركّز على كلمة «لماذا» في إجاباتك — لا تكتفِ بقول "استخدمنا Firebase" بل قل "اخترنا Firebase لأن...".
 2. إذا سُئِلت عن شيء لا تعرفه، قل "هذه نقطة ممتازة، يمكن تحسينها في التطوير المستقبلي" بدلاً من الاختلاق.
 3. أظهر التطبيق عملياً أثناء المناقشة إن أمكن — Demo أفضل من ألف كلمة.
 4. اذكر التحديات التي واجهتك بصدق — يظهر خبرتك الحقيقية.
- بالتوفيق في مناقشتك! 🎓